M

MANUAL TÉCNICO

Arquitectura y desarrollo

Cómo está construido MedCloud, qué tecnologías usa, dónde vive la lógica de negocio, cómo instalarlo y cómo extenderlo.

PRODUCTO	VERSIÓN	FECHA
MedCloud · Gestión clínica integral	React 19 · Node · SQL Server	Noviembre 2026

Contenido

1	Visión general INTRODUCCIÓN	2
2	Arquitectura ARQUITECTURA	3
3	Instalación INSTALACIÓN	5
4	Base de datos BASE DE DATOS	8
5	API API	10
6	Módulos MÓDULOS	14
A	Historial de cambios HISTORIAL	18

Datos de salud y base productiva

MedCloud guarda datos de salud (Ley 25.326 y Ley 26.529). La base del servidor local es productiva: no se prueba contra ella y al levantar el backend se envían mails reales. Este documento no incluye contraseñas ni claves: solo el nombre de cada variable y dónde se configura.

1 INTRODUCCIÓN

Visión general

MedCloud es un sistema web de gestión para clínicas y consultorios: pacientes, agenda de turnos, historia clínica, caja diaria, cobros y liquidación de honorarios médicos. Lo usan tres roles: **ADMIN**, **RECEPCION** y **MEDICO**.

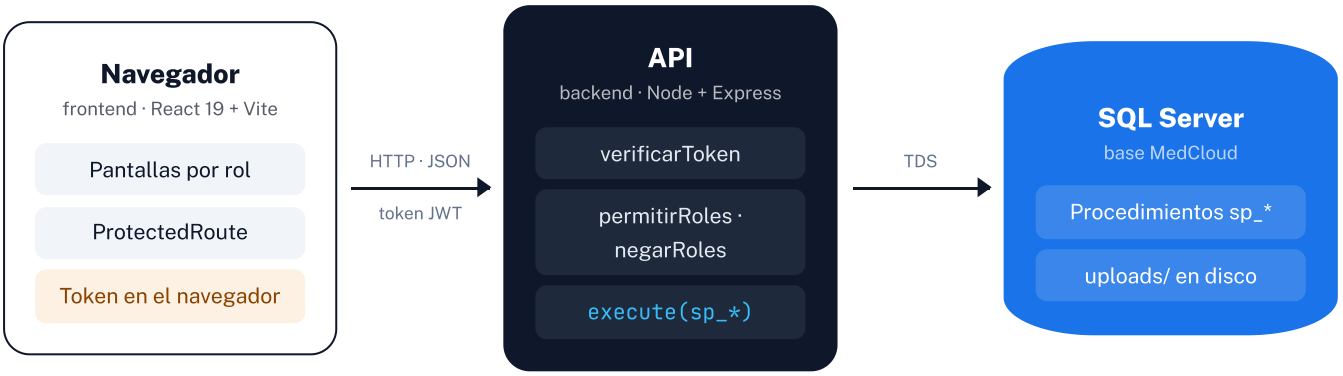


Figura 1. Capas del sistema. La API valida sesión y rol en cada ruta, ejecuta el procedimiento almacenado y responde.

El sistema tiene tres capas: un frontend en React (Vite) que corre en el navegador, una API en Node + Express y una base SQL Server donde vive la mayor parte de la lógica en procedimientos almacenados (`sp_*`). La API valida la sesión (JWT) y el rol en cada ruta antes de llamar al procedimiento.

2 ARQUITECTURA

Arquitectura

MedCloud separa navegador, API y base de datos. La API es delgada: recibe el pedido, valida sesión y rol, ejecuta un procedimiento almacenado y responde.

Carpetas

```
MedCloud-Backend/
  src/index.js              arranque: rutas, archivos de /uploads, scheduler de recordatorios
  src/config.js             configuración sensible del .env (JWT_SECRET); si falta, no arranca
  src/db.js                 pool de conexión a SQL Server
  src/middlewares/          auth.middleware.js: verificarToken, permitirRoles, negarRoles
  src/routes/               definición de endpoints y permisos por ruta
  src/controllers/          llaman a los procedimientos almacenados
  src/utils/                emailSender, scheduler, reciboHelper, phoneFormatter, roles
  uploads/                  estudios, comprobantes y recibos (no se versiona)
frontend/
  src/main.jsx              axios global: agrega el token y la URL del backend (API_URL)
  src/App.jsx               rutas, ProtectedRoute por rol, sesión vencida, inactividad, trial
  src/config.js             API_URL
  src/utils/                auth.js (usuario y rol), archivos.js (abrir archivos con sesión)
  src/components/           una pantalla por componente
database/DER.md             modelo de datos documentado
docs/                       manuales de usuario y técnico
```

Sesión

1. `POST /login` ejecuta `sp_LoginUsuario`, compara la contraseña y firma un JWT (8 h) con `id`, `username`, `rol`, `trial_dias_restantes` y un `session_id` aleatorio que también se guarda en `usuarios.session_token`.
2. El frontend guarda el token en `localStorage` y el interceptor de `main.jsx` lo envía como `Authorization: Bearer <token>` en cada pedido.
3. `verificarToken` valida firma y vencimiento con `JWT_SECRET`, compara `session_id` con el de la base (sesión única: si el usuario entra en otro equipo, la sesión anterior recibe `401 CONCURRENT_LOGIN_DISPLACED`) y bloquea las escrituras si el trial está vencido (`403 TRIAL_EXPIRED`).
4. Cualquier `401` con sesión guardada borra la sesión en el navegador y lleva al ingreso con el aviso "Sesión vencida" (o "Sesión finalizada" si fue desplazada). También se cierra a los 15 minutos sin actividad.

Permisos

- `permitirRoles(...)` deja pasar solo a los roles indicados; `negarRoles(...)` bloquea a los indicados. Se usan después de `verificarToken`.
- Regla del proyecto: **el backend replica lo que muestra el frontend**. Si una pantalla es solo para ADMIN y RECEPCION (`ProtectedRoute allowedRoles`), su API usa `permitirRoles('ADMIN', 'RECEPCION')`; si un botón se oculta al MEDICO, su API usa `negarRoles('MEDICO')`. La matriz completa está en api.md.

- Escalada de privilegios: aunque RECEPCION administra usuarios y roles, solo un ADMIN puede crear, editar o borrar usuarios con rol ADMIN, asignar ese rol o tocar el rol ADMIN (`src/utils/roles.js`).
- Historia clínica: solo MEDICO, y el médico se toma siempre del token (`req.user.id`), nunca de la URL o del cuerpo.

Archivos subidos

Los archivos de `uploads/` (estudios, comprobantes de transferencia, recibos de efectivo generados en HTML) se sirven solo con sesión en `GET /uploads/:archivo` . El nombre se reduce con `path.basename` para que no se pueda salir de la carpeta. Si un recibo de efectivo no existe en disco, se regenera a partir del pago. En el frontend, `utils/archivos.js` los descarga con el token y los abre en una pestaña nueva (la pestaña se abre en el mismo clic para que el navegador no la bloquee).

Decisiones de diseño

- **Sin edición ni borrado de evoluciones clínicas**: la historia clínica es inviolable (Ley 26.529, art. 18). Las rutas `PUT/DELETE /historial/:id` se quitaron; ninguna pantalla las usaba.
- **Falla temprana de configuración**: sin `JWT_SECRET` (mínimo 32 caracteres) el servidor no arranca, para que nunca funcione con una firma conocida.
- **Recordatorios**: `utils/scheduler.js` corre dentro del mismo proceso cada 10 minutos. Manda los recordatorios a los pacientes con turno el día siguiente y, a las 20 h, la agenda a cada médico. Usa una clave única por mensaje (`turno-recordatorio—<id>-<inicio>` , `agenda—<médico>-<fecha>`), así la mensajería no duplica aunque se reintente o haya dos instancias. Los errores definitivos (mail inválido, destinatario dado de baja, límite mensual) no se reintentan.

Mails

- **Ningún mail sale directo de MedCloud.** `utils/mensajeria.js` le pide cada envío a muchocodigo-mensajeria, que manda por Resend con reintentos, avisos de entrega y bajas. `utils/emailSender.js` arma los datos de cada plantilla `medcloud/*` : recordatorio, agenda, recuperar contraseña y soporte.
- **Datos de salud:** al paciente le llegan solo fecha, hora y profesional (sin especialidad ni motivo). La agenda que recibe el médico incluye el motivo de consulta, por decisión del responsable de MedCloud.
- **Confirmar o cancelar desde el mail:** el recordatorio trae dos links firmados (`utils/enlacesTurno.js`). El link abre una página (`utils/paginaTurno.js` , sin scripts y con los datos escapados) y recién el botón de esa página (POST) cambia el estado. Así un antivirus que abre los links no confirma ni cancela nada.
- **Hora argentina:** `utils/horaArgentina.js` muestra fechas y horas en `America/Argentina/Buenos_Aires` y calcula "mañana" como rango de instantes, sin depender de la zona del servidor. Las horas que carga el formulario se interpretan en la zona del proceso: en Docker hace falta `TZ=America/Argentina/Buenos_Aires` .

3 INSTALACIÓN

Instalación

Requisitos:

- Node 20 o superior.
- SQL Server con la base `MedCloud` y sus procedimientos `sp_*`.
- Para mandar mails, el servicio **muchocodigo-mensajeria** (repo aparte, envía por Resend) con un cliente dado de alta para la clínica.

Variables de entorno del backend

Se configuran en `MedCloud-Backend/.env` en desarrollo, o en el `environment:` del `docker-compose.yml` del servidor. La plantilla es `MedCloud-Backend/.env.example`.

VARIABLE	OBLIGATORIA	USO
`PORT`	no	Documentada; hoy el servidor escucha fijo en 3000
`DB_HOST`, **`DB_PORT`**, **`DB_NAME`**, **`DB_USER`**, **`DB_PASSWORD`**, **`DB_ENCRYPT`**	sí	Conexión a SQL Server (`src/db.js`)
`JWT_SECRET`	**sí**	Firma de las sesiones y de los links de turnos. Mínimo 32 caracteres; sin ella el servidor no arranca
`TZ`	**sí en Docker**	`America/Argentina/Buenos_Aires` . Las horas que carga el formulario de turnos se interpretan en la zona del proceso; los contenedores vienen en UTC y sin esto los turnos quedan corridos 3 horas
`MENSAJERIA_URL`	para mails	URL del servicio de mensajería. En el servidor: `http://mensajeria-api:3100` (red Docker `muchocodigo`)
`MENSAJERIA_CLAVE`	para mails	Clave del cliente (la clínica) en la mensajería (`mc_...`). Sin `MENSAJERIA_URL` y `MENSAJERIA_CLAVE` , MedCloud funciona pero no manda mails
`SOPORTE_EMAIL`	para soporte	Casilla que recibe los pedidos de la pantalla Soporte
`FRONTEND_URL`	sí	Base de los links de recuperación de contraseña. Sin ella, recuperar la contraseña falla
`BACKEND_URL`	sí	Base de los links firmados para confirmar o cancelar turnos desde el mail

Para generar un `JWT_SECRET` :

```
node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"
```

Atención

Cambiar `JWT_SECRET` cierra todas las sesiones abiertas e invalida los links de turnos de los mails ya enviados.

Mensajería

MedCloud no manda mails por su cuenta: le pide cada envío a muchocodigo-mensajeria (`POST /v1/mensajes`) con una plantilla `medcloud/*` y sus datos. Los textos de los mails viven en el repo de la mensajería (`plantillas/medcloud/`).

En el servidor:

1. La mensajería y MedCloud se hablan por la red Docker `muchocodigo` . El servicio `backend` del compose de MedCloud tiene que estar en esa red, además de `medcloud-network` .
2. En la mensajería se da de alta el sistema `medcloud` (remitente, por ejemplo `turnos@avisos.test.medcloud.ar`), un cliente por clínica y su clave. Los comandos están en el manual técnico de la mensajería.
3. Se cargan las plantillas con `node scripts/plantillas-cargar.js --sistema medcloud` .

Frontend

`frontend/.env.production` define la URL del backend (`REACT_APP_API_URL` ; también se acepta `VITE_API_URL`). Si no está, `src/config.js` usa `https://api.medcloud.ar` fuera de localhost y `http://localhost:3000` en desarrollo.

Puesta en marcha (desarrollo)

```
npm --prefix MedCloud-Backend install
npm --prefix MedCloud-Backend run dev
npm --prefix frontend install
npm --prefix frontend run dev
```

Atención

Al arrancar, el backend inicia el scheduler de recordatorios. Si `MENSAJERIA_URL` y `MENSAJERIA_CLAVE` apuntan a la mensajería real, manda mails a los pacientes de la base configurada. En desarrollo, dejalas vacías o usá la mensajería con el proveedor `consola` .

Despliegue

1. Antes de actualizar el código, revisar el compose del servidor:
   - que estén `JWT_SECRET` y `TZ` , porque sin ellos el backend no arranca o guarda mal las horas;
   - y, para mails, `MENSAJERIA_URL` , `MENSAJERIA_CLAVE` y `SOPORTE_EMAIL` .
2. `git pull` y `sudo docker compose up -d --build backend frontend` .

3 Verificar el ingreso con un usuario de cada rol.

Frontend y backend deben actualizarse juntos.

4 BASE DE DATOS

Base de datos

SQL Server, base `MedCloud`. El modelo está documentado en `database/DER.md`.

> **Atención**
> La mayoría de los procedimientos **existen solo en la base**. Se versionan en `database/procedures/` a medida que se tocan (un archivo por objeto, `CREATE OR ALTER`, como en StockyApp); reglas y cómo aplicarlos en `database/README.md`. Versionados: `sp_GetTurnos`.

Procedimientos que usa el backend

MÓDULO	PROCEDIMIENTOS
Sesión	`sp_LoginUsuario`, `sp_ChangePassword`, `sp_GetUsuarioByEmail`, `sp_CreatePasswordResetToken`, `sp_ValidatePasswordResetToken`, `sp_ResetPasswordByToken`
Usuarios y roles	`sp_GetUsuarios`, `sp_GetUsuarioById`, `sp_CreateUsuario`, `sp_SetUsuario`, `sp_DeleteUsuario`, `sp_GetRoles`, `sp_CreateRol`, `sp_SetRoles`, `sp_DeleteRol`
Administrativos	`getAdministrativos`, `createAdministrativo`, `setAdministrativo`, `deleteAdministrativo`
Profesionales	`sp_GetProfesionales`, `sp_GetProfesional`, `sp_CreateProfesional`, `sp_SetProfesional`, `sp_DeleteProfesional`, `sp_GetEspecialidades`
Pacientes	`sp_GetPacientes`, `sp_GetPaciente`, `sp_CreatePaciente`, `sp_SetPaciente`, `sp_DeletePaciente`, `sp_GetObrasSociales`, `sp_GetHistorialAsistenciaPaciente`
Historia clínica	`sp_GetHistorialesClinicos`, `sp_CreateHistorialClinico`, `sp_Set_CompartirHistorial`
Archivos	`sp_UploadArchivo`, `sp_GetArchivosPaciente`
Turnos	`sp_GetTurnos`, `sp_CreateTurno`, `sp_SetTurno`, `sp_DeleteTurno`, `sp_GetHorarios`, `sp_RegistrarPagoTurno`, `sp_CambiarEstadoTurno`
Caja y pagos	`sp_AbrirCaja`, `sp_CerrarCaja`, `sp_GetLiquidacionProfesional`, `sp_GetHistorialPagosPaciente`, `sp_GetHistorialPagosProfesional`, `sp_RegistrarPagoProfesional`

También hay consultas SQL armadas en el código (pendientes de pasar a procedimientos): caja activa e historial (`caja.controllers.js`), recordatorios, agenda de médicos y cambio de estado de turnos, tanto desde la agenda como desde el link del mail (`turnos.controllers.js`, `utils/scheduler.js`; usan `UPDATE Turnos SET estado` con la condición en el mismo `UPDATE`), regeneración de recibos (`index.js`), control de sesión única (`auth.middleware.js`) y guardado de `session_token` en el login.

Supuestos del código sobre la base

- `sp_GetRoles` devuelve `id` y `codigo`. Los códigos de rol usados son `ADMIN`, `RECEPCION` y `MEDICO`; `src/utils/roles.js` los usa para impedir la escalada de privilegios.
- `sp_GetUsuarioById` devuelve `rol_id`.
- `sp_GetHistorialesClinicos` y `sp_CreateHistorialClinico` reciben `usuario_id` (el usuario del token) y resuelven el profesional y la privacidad dentro del procedimiento.
- `usuarios.session_token` guarda la sesión activa (la columna se crea en el primer login si no existe).
- Estados de turno (`CK_Turnos_Estado`): `Pendiente`, `Confirmado`, `Completado`, `Cancelado`. `sp_GetTurnos` pasa a Completado los Confirmados que ya terminaron.
- `Pagos` tiene a lo sumo un pago por turno (`sp_RegistrarPagoTurno` lo valida). El pago toma la caja con `estado = 'Abierta'` y `fecha = CAST(GETDATE() AS DATE)`; el contenedor de SQL está en UTC, así que después de las 21 h de Argentina esa fecha ya es la del día siguiente (pendiente de revisar).
- `profesionales.sexo` (`Femenino` / `Masculino` / otros) define "Dra." o "Dr." en los mails; `turnos.recordatorio_dia_anterior_enviado` marca el recordatorio ya encolado; `envios_agenda_medico` registra la agenda enviada por médico y fecha.
- Las fechas de los turnos son instantes. El driver `mssql` los lee como UTC y MedCloud los muestra en hora argentina. Las cargas desde el formulario se interpretan en la zona del proceso (por eso `TZ` en Docker).

Pendiente de verificar con acceso de solo lectura

- Si `sp_GetUsuarios` o `sp_GetUsuarioById` devuelven `password_hash`.
- Si `sp_SetProfesional` conserva horarios y DNI (el backend no los envía al editar).
- Si `sp_Set_CompartirHistorial` valida que quien comparte sea el autor de la evolución.
- Columna de especialidad vigente en `profesionales` (`especialidad` en texto o `especialidad_id`).

5 API

API

Todas las rutas, salvo las públicas indicadas, requieren `Authorization: Bearer <token>`.

Roles: **A** = ADMIN, **R** = RECEPCION, **M** = MEDICO, **Todos** = cualquier usuario con sesión. "Todos menos M" significa que cualquier rol excepto MEDICO puede usarla.

Errores comunes de sesión y permisos:

CÓDIGO	CUÁNDO
`401`	Falta el token, es inválido o venció (`Token inválido o expirado`)
`401` `CONCURRENT_LOGIN_DISPLACED`	El usuario inició sesión en otro equipo
`403`	El rol no tiene permiso (`No tenés permisos para realizar esta acción.`)
`403` `TRIAL_EXPIRED`	Período de prueba vencido: solo se permiten lecturas (GET)
`500`	Error de base de datos (texto del error)

Públicas

MÉTODO	RUTA	USO
POST	`/login`	Ingreso. Cuerpo `{ username, password }` → `{ token, user }`
POST	`/auth/forgot-password`	Pide el mail de recuperación. Responde siempre el mismo mensaje
POST	`/auth/validate-reset-token`	Valida el link de recuperación
POST	`/auth/reset-password`	Cambia la contraseña con el link
GET	`/turnos/respuesta?t=<link firmado>`	Página para el paciente que tocó "Sí, voy a asistir" o "No puedo asistir" en el mail: muestra el turno y un botón. **No cambia nada** (los antivirus del correo abren los links)
POST	`/turnos/respuesta?t=<link firmado>`	Confirma o cancela el turno. Solo turnos Pendiente o Confirmado que todavía no empezaron. Responde una página: `400` link inválido, `404` turno inexistente, `409` no se puede modificar, `410` el turno ya pasó

El link firmado (`utils/enlacesTurno.js`) lleva el turno, la acción y el vencimiento (inicio del turno), firmado con una clave derivada de `JWT_SECRET`. No se puede adivinar ni reutilizar para otro turno.

Sesión

MÉTODO	RUTA	ROLES	USO
GET	`/auth/verify-session`	Todos	Control de sesión única (cada 30 s desde el frontend)

Usuarios

MÉTODO	RUTA	ROLES	NOTAS
GET	`/usuarios`, `/usuarios/:id`	A, R	
POST	`/usuarios`	A, R	Solo A puede crear con rol ADMIN (`403`)
PUT	`/usuarios/:id`	A, R	Solo A puede editar a un ADMIN o asignar el rol ADMIN (`403`)
DELETE	`/usuarios/:id`	A, R	Solo A puede borrar a un ADMIN (`403`)
PUT	`/usuarios/:id/password`	Todos	Solo la propia contraseña; A puede cambiar cualquiera (`403 Solo podés cambiar tu propia contraseña.`)

Roles

MÉTODO	RUTA	ROLES	NOTAS
GET	`/roles`	A, R	
POST / PUT / DELETE	`/roles`, `/roles/:id`	A, R	Solo A puede crear, editar o borrar el rol con código ADMIN (`403`)

Profesionales

MÉTODO	RUTA	ROLES
GET	`/profesionales`, `/profesionales/:id`, `/especialidades`	Todos
POST / PUT / DELETE	`/profesionales`, `/profesionales/:id`	A, R

Administrativos

MÉTODO	RUTA	ROLES	NOTAS
GET / POST / PUT / DELETE	`/administrativos`, `/administrativos/:id`	A	Ninguna pantalla las usa

Pacientes

MÉTODO	RUTA	ROLES	NOTAS
GET	`/pacientes`	Todos	`sp_GetPacientes` filtra por usuario (el médico ve los suyos)
GET	`/pacientes/:id`	Todos	
GET	`/pacientes/:id/asistencia`	Todos menos M	
POST / PUT	`/pacientes`, `/pacientes/:id`	A, R	
DELETE	`/pacientes/:id`	A, R	Responde `204`
GET	`/obras-sociales`	Todos	

Historia clínica

El médico se identifica siempre por el token, nunca por la URL ni por el cuerpo.

MÉTODO	RUTA	ROLES	NOTAS
GET	`/historial/paciente/:paciente_id`	M	
GET	`/historial/paciente/:paciente_id/profesional/:x`	M	Ruta anterior; `:x` se ignora
POST	`/historial`	M	`{ paciente_id, turno_id, motivo, evolucion, diagnostico, tratamiento }`
POST	`/historial/compartir`	M	`{ historial_id, profesional_invitado_id }`

No existen `PUT` ni `DELETE`: las evoluciones no se modifican ni se borran.

Archivos

MÉTODO	RUTA	ROLES	NOTAS
POST	`/archivos`	A, R	`multipart/form-data`, campo `archivo`, más `paciente_id` y `turno_id`
GET	`/archivos/:paciente_id`	Todos	Lista de estudios del paciente
GET	`/uploads/:archivo`	Todos	Descarga el archivo; regenera recibos de efectivo faltantes. `404` si no existe

Turnos

MÉTODO	RUTA	ROLES
GET	`/turnos` , `/horarios`	Todos (`/turnos` incluye `pagado` y `metodo_pago` por turno)
POST	`/turnos`	Todos menos M
PUT	`/turnos/:id`	Todos
DELETE	`/turnos/:id`	Todos
PUT	`/turnos/:id/estado`	Todos menos M
POST	`/turnos/:id/pagar`	Todos menos M
POST	`/turnos/pagar-multiples`	Todos menos M
POST	`/turnos/:id/enviar-email`	Todos menos M

Caja y liquidaciones

MÉTODO	RUTA	ROLES
GET	`/caja/activa` , `/caja/historial`	A, R
POST	`/caja/abrir` , `/caja/cerrar`	A, R
GET	`/liquidaciones?profesional_id=&fecha_desde=&fecha_hasta=`	A, R
GET	`/pagos/profesional/:id`	A, R
POST	`/pagos/profesional/:id`	A, R
GET	`/pagos/paciente/:id`	Todos menos M

Soporte

MÉTODO	RUTA	ROLES
POST	`/api/soporte`	Todos

6 MÓDULOS

Módulos

Cada módulo tiene su pantalla en `frontend/src/components/` y su API en `MedCloud-Backend/src/routes/` y `src/controllers/`. Los permisos de cada endpoint están en api.md.

Sesión y seguridad

Estado: en uso · **Permisos:** todos

Archivos

- Frontend: `LoginPage.jsx`, `ForgotPassword.jsx`, `ResetPassword.jsx`, `ProtectedRoute.jsx`, `App.jsx`, `utils/auth.js`, `utils/useInactivityTimer.js`
- Backend: `controllers/auth.controllers.js`, `controllers/passwordReset.controllers.js`, `middlewares/auth.middleware.js`, `config.js`, `utils/roles.js`

Reglas

- Sesión JWT de 8 h firmada con `JWT_SECRET`; sesión única por usuario; cierre por 15 minutos de inactividad.
- Cambio de contraseña obligatorio en el primer ingreso: el frontend envía el token recibido en el login de forma explícita, porque todavía no está guardado.
- Un `401` cierra la sesión en el navegador y lleva al ingreso.

Usuarios y roles

Estado: en uso · **Permisos:** ADMIN y RECEPCION

Archivos

- Frontend: `UsuariosList.jsx`, `UsuariosForm.jsx`, `RolList.jsx`, `RolForm.jsx`
- Backend: `routes/usuarios.routes.js`, `routes/roles.routes.js`, `controllers/usuarios.controllers.js`, `controllers/roles.controllers.js`

Reglas

- Solo un ADMIN gestiona usuarios ADMIN o el rol ADMIN (evita que recepción se cree un administrador).

Pacientes

Estado: en uso · **Permisos:** listado y ficha para todos; alta, edición y baja para ADMIN y RECEPCION; asistencia y pagos ocultos al MEDICO

Archivos

- Frontend: `PacientesList.jsx`, `PacientesForm.jsx`
- Backend: `routes/pacientes.routes.js`, `controllers/pacientes.controllers.js`, `routes/archivos.routes.js`, `controllers/archivos.controllers.js`

Reglas

- El teléfono se normaliza a `549…` (`utils/phoneFormatter.js`).

- Los estudios se suben a `uploads/` y se ven solo con sesión.

Historia clínica

Estado: en uso · **Permisos:** solo MEDICO

Archivos

- Frontend: `HistorialClinico.jsx`
- Backend: `routes/historial.routes.js`, `controllers/historial.controllers.js`

Reglas

- El médico se toma del token. Las evoluciones no se editan ni se borran (Ley 26.529).

Profesionales

Estado: en uso · **Permisos:** consulta para todos; gestión para ADMIN y RECEPCION

Archivos

- Frontend: `ProfesionalList.jsx`, `ProfesionalForm.jsx`
- Backend: `routes/profesionales.routes.js`, `controllers/profesionales.controllers.js`

Reglas

- La pantalla carga el % de honorario del médico y el backend guarda la retención de la clínica (`100 - honorario`).

Turnos

Estado: en uso · **Permisos:** agenda para todos; agendar, confirmar, cobrar y enviar mails para todos menos MEDICO

Archivos

- Frontend: `TurnosList.jsx`, `TurnosForm.jsx`
- Backend: `routes/turnos.routes.js`, `controllers/turnos.controllers.js`, `utils/emailSender.js`, `utils/scheduler.js`, `utils/reciboHelper.js`, `utils/enlacesTurno.js`, `utils/paginaTurno.js`, `utils/horaArgentina.js`, `utils/mensajeria.js`

Reglas

- **Confirmado ya no significa pagado.** Confirmado = el paciente va a asistir (botón ✔ Confirmar, link del mail o cobro). Pagado = existe una fila en `Pagos` para el turno (`sp_GetTurnos` devuelve `pagado` y `metodo_pago`). La agenda usa `estaPagado` / `sePuedeCobrar` (`TurnosList.jsx`): Cobrar y la casilla de cobro múltiple aparecen si no hay pago y el turno no está Cancelado, también en Completado (cobro después de la consulta).
- El cobro registra el pago en la caja abierta y confirma el turno. En efectivo se genera un recibo HTML en `uploads/`; con transferencia o MercadoPago se adjunta el comprobante. Tarjeta y obra social no tienen comprobante. `sp_RegistrarPagoTurno` rechaza un segundo pago del mismo turno.
- El día anterior, cada paciente recibe un recordatorio con dos botones: confirmar o cancelar. El paciente solo puede cambiar turnos Pendiente o Confirmado que todavía no empezaron. Un turno Cancelado no se puede volver a confirmar desde el mail.
- `PUT /turnos/:id/estado` valida que el estado sea uno de los cuatro permitidos.

Mails

Estado: en uso (requiere la mensajería instalada) · **Permisos:** automático y botón Mail de la agenda (todos menos MEDICO)

Archivos

- Backend: `utils/mensajeria.js` (cliente), `utils/emailSender.js` (datos de cada plantilla), `utils/scheduler.js`, `controllers/passwordReset.controllers.js`, `routes/soporte.routes.js`
- Plantillas: repo muchocodigo-mensajeria, `plantillas/medcloud/` (`recordatorio-turno`, `agenda-medico`, `recuperar-clave`, `soporte`)

Reglas

- Sin `MENSAJERIA_URL` y `MENSAJERIA_CLAVE`, el scheduler no envía y lo avisa al arrancar. El botón Mail y el soporte responden un error.
- El link de recuperación sale siempre de `FRONTEND_URL`, nunca de los encabezados del pedido.
- La agenda del médico se recorta a ~3.800 caracteres, con el aviso "y N turnos más (ver MedCloud)". La mensajería admite hasta 4.000 por dato.

Caja diaria y liquidaciones

Estado: en uso · **Permisos:** ADMIN y RECEPCION

Archivos

- Frontend: `CajaControl.jsx`, `CajaHistorial.jsx`, `Liquidaciones.jsx`
- Backend: `routes/caja.routes.js`, `controllers/caja.controllers.js`, `routes/pagos.routes.js`, `controllers/pagos.controllers.js`

Cómo se probó (avance del 2026-11-02: confirmar asistencia y Pagado)

- Se leyó en producción (solo la definición, sin datos) `sp_GetTurnos` y `sp_RegistrarPagoTurno`: la base ya impedía el doble cobro; el problema era solo de pantalla.
- `npm run build` del frontend OK; lint sin problemas nuevos (los 3 avisos de `TurnosList.jsx` / `TurnosForm.jsx` ya existían).
- **Falta probar** con el `sp_GetTurnos` nuevo aplicado: cobrar con tarjeta y ver Pagado sin botón Cobrar; cobrar un turno Confirmado sin pago; que el MEDICO no vea Pagado.

Cómo se probó (avance del 2026-10-13: mails por la mensajería)

- Prueba con base y mensajería simuladas, sin conexión real ni envíos: 34 casos OK.
  - **Links firmados:** se verifican; se rechazan si se alteran, si se cambia el turno o si están mal formados; vencen.
  - **Hora argentina:** "mañana" va de 03:00 a 03:00 UTC, y un turno de 13:30 UTC se muestra 10:30.
  - **Plantillas:** cada pedido trae exactamente los datos que pide su plantilla (se leyeron los JSON reales de `plantillas/medcloud/`). El recordatorio va sin motivo ni especialidad. La agenda incluye el motivo y se recorta.
  - **Recuperar contraseña:** usa `FRONTEND_URL` e ignora el `Origin`. Con un email inexistente da la misma respuesta y no se envía nada.

    - **Página del paciente:** el GET no cambia el estado; el POST confirma o cancela. Turno ya pasado: 410. Completado o Cancelado: 409. Link inválido: 400. Un nombre con HTML se muestra escapado y la página no tiene scripts. La ruta vieja `/turnos/accion-publica` ya no existe.
    - **Recordatorios:** se buscan por el rango del día argentino.
- La prueba de permisos del paso 1 sigue OK, incluida `PUT /turnos/:id/estado` (MEDICO: 403).
- **Falta probar** con la mensajería instalada en el servidor: envío real a una casilla propia y respuesta desde el celular.

Cómo se probó (avance del 2026-10-10)

- Prueba aislada de permisos con las rutas reales, tokens de cada rol y una base inexistente (sin scheduler ni mails): 29 rutas × 4 casos (sin sesión, MEDICO, RECEPCION, ADMIN) dan 401/403/acceso según la matriz de api.md; `PUT` / `DELETE /historial/:id` dan 404; un nombre de archivo con `../` se reduce a un nombre dentro de `uploads/`.
- Bloqueo de escalada con base simulada: 16 casos (crear, editar y borrar usuarios ADMIN y el rol ADMIN siendo RECEPCION o ADMIN) dan el resultado esperado.
- `node --check` de todo el backend, `npm run build` del frontend y lint sin problemas nuevos.
- **No se probó contra la base real** (no hay acceso todavía): falta recorrer en el navegador el ingreso, el cambio de contraseña inicial, la apertura de estudios y recibos y el borrado de pacientes con un usuario de cada rol.

A HISTORIAL

Historial de cambios

2026-11-02 — Agenda: confirmar asistencia y "Pagado" con cualquier método de pago

- **Qué cambió:**
  - Botón ✔ Confirmar en la agenda (`PUT /turnos/:id/estado`); el formulario de turno vuelve a permitir elegir Confirmado. Confirmado ya no significa pagado.
  - "Pagado" sale del pago registrado y no del comprobante: los turnos cobrados con tarjeta u obra social ya no ofrecen Cobrar (antes el botón quedaba y la base rechazaba el segundo cobro con "Esta consulta ya fue cobrada previamente por caja").
  - Se puede cobrar un turno Confirmado o Completado sin pago (cobro después de la consulta), también en el cobro múltiple.
  - La etiqueta Pagado no se muestra al MEDICO.
- **Módulos:** turnos.
- **Backend/API:** sin cambios de código; `GET /turnos` devuelve `pagado` y `metodo_pago` .
- **Base de datos:** `sp_GetTurnos` versionado en `database/procedures/` (primero tal cual estaba en producción, después el cambio) y nuevo `database/README.md` . El cambio solo agrega columnas: se aplica antes de desplegar el frontend.
- **Configuración:** ninguna.
- **Pruebas:** ver modulos.md.
- **Commits:** ver el PR de la rama `turnos/confirmar-asistencia-y-pagado` .

2026-10-13 — Mails por la mensajería y confirmar o cancelar turnos desde el mail

- **Qué cambió:**
  - MedCloud deja de usar SMTP (Gmail): todos los mails salen por muchocodigo-mensajeria con las plantillas `medcloud/*` .
  - El recordatorio al paciente trae links firmados para confirmar o cancelar, con una página intermedia de confirmación y validación de estado y horario. Reemplaza la ruta pública por id (`/turnos/accion-publica`), que nunca llegó a producción.
  - El recordatorio ya no muestra especialidad.
  - La agenda del médico incluye el motivo de consulta.
  - Fechas y horas en hora argentina, sin depender de la zona del servidor.
  - El link de recuperación de contraseña sale solo de `FRONTEND_URL` , que ya no se arma con el `Origin` del pedido: cierra el riesgo de link apuntando a otro sitio.
  - Los logs de recuperación ya no muestran emails.
- **Módulos:** turnos, mails, sesión (recuperar contraseña), soporte.
- **Backend/API:**
  - Nuevas: `GET` / `POST /turnos/respuesta` (públicas, protegidas por el link firmado) y `PUT /turnos/:id/estado` (todos menos MEDICO, con validación del estado).
  - Quitada: `/turnos/accion-publica` .

- Nuevos `utils/mensajeria.js`, `utils/enlacesTurno.js`, `utils/paginaTurno.js` y `utils/horaArgentina.js`.
- Se quitaron `nodemailer` y el script `test-notifs.js`.

- **Base de datos:** sin objetos nuevos. Se lee `profesionales.sexo` (Dr./Dra.) y el estado se cambia con un `UPDATE` condicionado.
- **Configuración:**
  - Nuevas: `MENSAJERIA_URL`, `MENSAJERIA_CLAVE`, `SOPORTE_EMAIL` y `TZ=America/Argentina/Buenos_Aires`.
  - Se quitan las `SMTP_*`.
  - `FRONTEND_URL` pasa a ser obligatoria para recuperar la contraseña.
  - El backend tiene que estar en la red Docker `muchocodigo`.
- **Pruebas:** ver modulos.md.
- **Commits:** ver el PR de la rama `mails/mensajeria`.

2026-10-10 — Seguridad: sesión y permisos en todas las rutas

- **Qué cambió:**
  - Todas las rutas de la API exigen sesión, salvo el login, la recuperación de contraseña y el link público de turnos.
  - Los permisos por rol replican los del frontend.
  - Solo un ADMIN gestiona usuarios o el rol ADMIN.
  - El cambio de contraseña solo permite la propia (o cualquiera si sos ADMIN).
  - La historia clínica toma al médico del token, y se quitaron la edición y el borrado de evoluciones.
  - Los archivos de `uploads/` se sirven solo con sesión y sin poder salir de la carpeta.
  - El secreto de las sesiones pasa al `.env` (`JWT_SECRET`).
  - `jsonwebtoken` queda declarada como dependencia.
  - Se corrigió el borrado de pacientes, que nunca respondía.
  - Se corrigieron los links de recibos que apuntaban a `localhost`.
  - La sesión vencida ahora lleva al ingreso.
- **Módulos:** sesión, usuarios, roles, profesionales, administrativos, pacientes, historia clínica, archivos, turnos, caja, liquidaciones.
- **Backend/API:**
  - Nuevos: `GET /historial/paciente/:id`, `src/config.js`, `src/utils/roles.js`, y `permitirRoles` / `negarRoles` en `auth.middleware.js`.
  - Eliminados: `PUT/DELETE /historial/:id`.
  - `GET /uploads/:archivo` ahora requiere sesión.
  - Falta de token: ahora `401` (antes `403`).
  - `DELETE /pacientes/:id` responde `204`.
- **Frontend:**
  - Nuevo `utils/archivos.js`.
  - Estudios y recibos se abren con sesión desde `PacientesForm`, `PacientesList`, `HistorialClinico`, `Liquidaciones` y `TurnosList`.
  - El cambio de contraseña inicial envía el token.
  - Aviso de "Sesión vencida" en `App.jsx`.

- **Base de datos:** sin cambios (se usan `sp_GetRoles` y `sp_GetUsuarioById` para el control de escalada).
- **Configuración:** nueva variable obligatoria `JWT_SECRET` y nuevo `MedCloud-Backend/.env.example`. Al desplegar se cierran todas las sesiones una vez.
- **Pendiente:** el link público de confirmación/cancelación de turnos (`/turnos/accion-publica` , cambio todavía sin commitear) necesita un token firmado antes de subir a producción.
- **Pruebas:** ver modulos.md.
- **Commits:** `0048fc7` (ignorar archivos del servidor), `bc7dae9` (JWT_SECRET), `a9599f1` (sesión y roles en rutas), `63cf961` (borrar paciente), `88db6d9` (frontend: archivos con sesión y sesión vencida).